
Sebenta

Aula 02 — Sensores, Conversão AD/DA e Interrupções

Tecnologias de Comunicação para a Internet das Coisas

Ano letivo 2026–2027

Catarina Silva

ESTGA, Universidade de Aveiro

Texto de apoio ao estudo autónomo

Índice

Como Usar Esta Sebenta	2
1 Comentário aos Diapositivos	2
2. Agenda	2
3. Da Teoria à Prática	3
4. O que é um Sensor (1/2)	4
5. O que é um Sensor (2/2)	5
6. Porque Converter Analógico em Digital (1/2)	6
7. Porque Converter Analógico em Digital (2/2)	7
8. Amostragem (1/2)	8
9. Amostragem (2/2)	9
10. Resolução e Erro de Quantificação	10
11. Na Prática: o ADC do Arduino Uno	11
12. Porquê Interrupções?	12
13. Interrupções por Hardware (1/2)	13
14. Interrupções por Hardware (2/2)	14
15. Interrupções por Software (1/2)	15
16. Interrupções por Software (2/2)	16
17. Boas Práticas numa ISR	17
18. O Hardware de Hoje (1/2)	18
19. O Hardware de Hoje (2/2)	19
20. Cuidados de Montagem	20
21. Programação Visual: No-Code e Low-Code	21
22. Node-RED	22
23. Estrutura da Sessão Prática (2h)	23
24. Síntese da Aula	24
Verificação Final	25
Referências	25

Como Usar Esta Sebenta

Este texto acompanha, diapositivo a diapositivo, a exposição da Aula 02. Cada página reproduz um diapositivo e indica, por baixo, o que dele se deve retirar.

A leitura não substitui a aula nem os diapositivos. Destina-se ao estudo posterior, quando o estudante já ouviu a exposição e pretende consolidar, verificar se compreendeu o essencial ou preparar a avaliação. Os comentários explicitam ligações entre diapositivos que na exposição oral ficam implícitas, e assinalam os pontos em que a matéria costuma ser mal compreendida.

Sempre que um comentário formula uma pergunta, essa pergunta é o critério de verificação: se não conseguir respondê-la, convém voltar ao diapositivo antes de avançar.

1. Comentário aos Diapositivos

Diapositivo 2. Agenda

Agenda

- Sensores: do mundo analógico ao mundo digital
- Conversão analógico-digital: amostragem e resolução
- Interrupções por hardware e por software
- O hardware de hoje: Arduino Uno e os três sensores do guião
- Programação visual: um primeiro contacto com o Node-RED
- Estrutura da sessão prática (2h)

O percurso da aula. A exposição tem três blocos. O primeiro trata a aquisição de grandezas físicas, o segundo a resposta a eventos, o terceiro as ferramentas de programação visual. Os três convergem no mesmo ponto: preparar a sessão laboratorial que se segue imediatamente. Esta aula não é autónoma, é instrumental.

Diapositivo 3. Da Teoria à Prática

Da Teoria à Prática

- Na Aula 01 vimos a arquitetura de referência de IoT e o paradigma M2M: sensores captam grandezas do mundo real e alimentam uma cadeia de comunicação até à *cloud*.
- Hoje fechamos o primeiro elo dessa cadeia: como um microcontrolador lê um sensor e como reage a eventos sem estar permanentemente ocupado à espera deles.
- Estes 45 minutos preparam diretamente os cinco exercícios do guião prático desta aula.

O que já foi dado e o que falta. Na Aula 01 estudou-se a arquitetura de referência e o paradigma máquina a máquina. Ficou estabelecido que os sensores captam grandezas e alimentam uma cadeia de comunicação. Falta fechar o primeiro elo dessa cadeia: como é que um microcontrolador lê efetivamente um sensor, e como reage a acontecimentos sem ficar permanentemente ocupado à espera deles.

Diapositivo 4. O que é um Sensor (1/2)

O que é um Sensor (1/2)

- Um sensor converte uma grandeza física (temperatura, humidade, distância, luz, ...) num sinal elétrico que um sistema digital consegue processar.
- Pode ser simples, dedicado a uma única grandeza, ou complexo, combinando várias leituras e algum pré-processamento.
- É o termo genérico por detrás daquilo a que chamamos informalmente “as coisas” de um sistema IoT.

O que é um sensor. Um sensor converte uma grandeza física num sinal elétrico que um sistema digital consegue processar. Pode ser simples, dedicado a uma grandeza, ou complexo, combinando leituras e algum pré-processamento. É o termo técnico por detrás daquilo a que informalmente se chama as coisas de um sistema IoT.

Diapositivo 5. O que é um Sensor (2/2)

O que é um Sensor (2/2)

- **Sensores digitais:** já entregam uma leitura discreta ou um protocolo próprio (ex.: o DHT11, que devolve valores por um barramento de um fio).
- **Sensores analógicos:** entregam uma tensão contínua, proporcional à grandeza medida (ex.: o sensor de humidade do solo, lido no pino A0).
- Os três sensores do guião de hoje cobrem exatamente estes dois casos, mais um sensor por tempo de eco (HC-SR04).

Digitais e analógicos. Os sensores digitais entregam uma leitura discreta ou usam um protocolo próprio. Os analógicos entregam uma tensão contínua, proporcional à grandeza medida. Esta distinção não é académica: determina se basta uma função de leitura direta ou se é preciso uma biblioteca dedicada. Os três sensores do laboratório cobrem os dois casos, mais um terceiro que mede tempo de eco.

Diapositivo 6. Porque Converter Analógico em Digital (1/2)

Porque Converter Analógico em Digital (1/2)

- Os microprocessadores são máquinas digitais: só manipulam sequências de bits.
- O mundo físico é essencialmente analógico: as grandezas variam de forma contínua no tempo.
- É necessário um circuito de interface entre os dois mundos.

A incompatibilidade de fundo. Os microprocessadores são máquinas digitais e só manipulam sequências de bits. O mundo físico é analógico e varia de forma contínua. Entre os dois é necessário um circuito de interface. Este diapositivo estabelece o problema; o seguinte apresenta a solução.

Diapositivo 7. Porque Converter Analógico em Digital (2/2)

Porque Converter Analógico em Digital (2/2)

- O **conversor analógico-digital** (ADC) transforma a tensão contínua do sensor numa sequência de números que o microcontrolador pode processar.
- O **conversor digital-analógico** (DAC) faz o caminho inverso, quando é o sistema digital a gerar um sinal contínuo.
- No guião de hoje só precisamos do ADC: é ele que está por detrás da função `analogRead()`.

ADC e DAC. O conversor analógico-digital transforma a tensão contínua numa sequência de números. O conversor digital-analógico faz o percurso inverso. No laboratório usa-se apenas o primeiro. É ele que está por detrás da função de leitura analógica do Arduino, o que convém reter para o exercício do sensor de humidade.

Diapositivo 8. Amostragem (1/2)

Amostragem (1/2)

- A amostragem consiste em medir o sinal de entrada em instantes periódicos, e não continuamente.
- O conjunto de valores medidos é apenas uma representação do sinal original — nunca o sinal exato.
- Pergunta central: com que frequência é preciso medir para não perder informação relevante?

O que é amostrar. A amostragem consiste em medir o sinal em instantes periódicos, e não continuamente. O conjunto de valores medidos é uma representação do sinal original, nunca o sinal exato. Daqui decorre a pergunta central: com que frequência é preciso medir para não perder informação relevante.

Diapositivo 9. Amostragem (2/2)

Amostragem (2/2)

- O **Teorema de Nyquist** responde: a frequência de amostragem f_s tem de ser pelo menos o dobro da componente de frequência mais rápida do sinal ($f_s = 2B$).
- Na prática usa-se uma margem confortável, tipicamente 5 a 10 vezes B .
- Para as grandezas físicas lentas deste guião (temperatura, humidade, distância), uma leitura por segundo é mais do que suficiente.

O critério de Nyquist. A frequência de amostragem tem de ser pelo menos o dobro da componente mais rápida do sinal. Na prática usa-se uma margem de cinco a dez vezes. Para as grandezas lentas do laboratório, temperatura, humidade e distância, uma leitura por segundo é largamente suficiente. O critério importa quando se medem grandezas rápidas, como som ou vibração.

Diapositivo 10. Resolução e Erro de Quantificação

Resolução e Erro de Quantificação

- Além de *quando* medir, importa *com que precisão*: a resolução é o número de bits usados para representar cada amostra.
- Mais bits → mais valores discretos possíveis → menor erro de quantificação, à custa de mais memória e mais tempo de conversão.
- Exemplo: 3 bits só distinguem 8 níveis; 4 bits já distinguem 16 — o dobro da precisão.

Quantos bits por amostra. Além de quando medir, importa com que precisão. A resolução é o número de bits usados para representar cada amostra. Mais bits significam mais níveis discretos e menor erro de quantificação, à custa de memória e de tempo de conversão. Três bits distinguem oito níveis, quatro bits distinguem dezasseis.

Diapositivo 11. Na Prática: o ADC do Arduino Uno

Na Prática: o ADC do Arduino Uno

- O Arduino Uno tem um ADC de 10 bits: cada leitura de `analogRead()` devolve um valor entre 0 e 1023.
- Esse valor é proporcional à tensão lida no pino analógico, entre 0V e a tensão de referência (5V, por omissão).
- É este o mecanismo que vamos usar já a seguir para ler o sensor de humidade do solo.

O caso concreto do Arduino. O Arduino Uno tem um conversor de 10 bits, pelo que cada leitura devolve um valor entre 0 e 1023. Esse valor é proporcional à tensão lida, entre zero volts e a tensão de referência. Quando no laboratório aparecer um número nessa gama, é esta a explicação. Dois elevado a dez são 1024 valores possíveis.

Diapositivo 12. Porquê Interrupções?

Porquê Interrupções?

- Até agora, o `loop()` do Arduino verifica o estado das coisas repetidamente: é o chamado *polling*.
- O *polling* funciona, mas desperdiça ciclos de processamento e pode “perder” um evento muito curto entre duas verificações.
- Uma **interrupção** inverte a lógica: o processador continua a sua tarefa e só é desviado quando o evento realmente acontece.

Sondagem e o seu custo. Até aqui, o ciclo principal do programa verifica repetidamente o estado das coisas. A esta abordagem chama-se sondagem. Funciona, mas desperdiça ciclos de processamento e pode perder um acontecimento curto ocorrido entre duas verificações. A interrupção inverte a lógica: o processador prossegue a sua tarefa e só é desviado quando o acontecimento ocorre.

Diapositivo 13. Interrupções por Hardware (1/2)

Interrupções por Hardware (1/2)

- Certos pinos digitais do Arduino Uno (2 e 3) estão ligados diretamente a um controlador de interrupções do microcontrolador.
- Uma mudança elétrica nesse pino — por exemplo, ao premir um botão — dispara de imediato uma **ISR** (*Interrupt Service Routine*), independentemente do que o `loop()` estava a fazer.
- É o mecanismo mais direto para reagir a eventos externos rápidos: um botão, um sensor de fim-de-curso, um pulso de outro circuito.

Interrupções ligadas a pinos. Certos pinos digitais do Arduino Uno, o 2 e o 3, estão ligados a um controlador de interrupções. Uma alteração elétrica nesse pino dispara de imediato uma rotina de serviço, independentemente do que o ciclo principal estivesse a fazer. É o mecanismo mais direto para reagir a acontecimentos externos rápidos.

Diapositivo 14. Interrupções por Hardware (2/2)**Interrupções por Hardware (2/2)**

- Em código: `attachInterrupt(pino, funcaoISR, modo)`, em que o modo define quando disparar (`CHANGE`, `RISING`, `FALLING`).
- No exercício de hoje, um botão ligado ao pino 2 dispara uma ISR que acende ou apaga o LED — sem que o `loop()` precise de verificar nada.
- Repara que o `loop()` do exemplo fica praticamente vazio: todo o trabalho acontece na ISR.

O que observar no exercício. A função de associação recebe o pino, a rotina e o modo, que define quando disparar. O detalhe a observar no laboratório é que o ciclo principal fica praticamente vazio. Todo o comportamento reside na rotina de serviço, e isso é visível no código.

Diapositivo 15. Interrupções por Software (1/2)

Interrupções por Software (1/2)

- Nem todos os eventos vêm de um pino: por vezes queremos apenas que algo aconteça **periodicamente**, de tempos a tempos.
- É o caso de um temporizador: em vez de contar manualmente o tempo no `loop()`, um periférico interno do microcontrolador conta por nós e dispara uma interrupção quando o tempo definido se esgota.
- No Arduino Uno isto é acessível através da biblioteca `TimerOne`.

Quando o evento não vem de fora. Nem todos os acontecimentos vêm de um pino. Por vezes pretende-se apenas que algo ocorra periodicamente. Um periférico interno conta o tempo e dispara uma interrupção quando o período se esgota, o que dispensa contagem manual no ciclo principal.

Diapositivo 16. Interrupções por Software (2/2)

Interrupções por Software (2/2)

- Em código: `Timer1.initialize(us)` define o período (em microssegundos) e `Timer1.attachInterrupt(funcao)` associa a ISR a esse temporizador.
- No exercício de hoje, uma ISR periódica de 10 em 10 segundos interfere com um contador que o `loop()` também está a incrementar — um bom exemplo de como duas “linhas de execução” partilham a mesma variável.
- Este exercício prepara diretamente o terreno para as leituras periódicas de sensores que se seguem.

Duas linhas de execução sobre a mesma variável. A biblioteca de temporizador permite definir o período e associar a rotina. No exercício, uma rotina periódica subtrai a um contador que o ciclo principal incrementa. É um exemplo deliberadamente simples de duas linhas de execução a atuar sobre a mesma variável, e prepara o diapositivo seguinte.

Diapositivo 17. Boas Práticas numa ISR

Boas Práticas numa ISR

- Uma ISR deve ser **curta e rápida**: enquanto executa, o resto do programa espera.
- Variáveis partilhadas entre a ISR e o `loop()` devem ser declaradas `volatile`, para o compilador não as guardar apenas em registos e assumir valores desatualizados.
- Evitar `delay()` e operações lentas dentro de uma ISR — é o erro mais comum de quem começa a usar interrupções.

As três regras. Uma rotina de serviço deve ser curta, porque enquanto executa o resto do programa espera. As variáveis partilhadas devem ser declaradas voláteis, para o compilador não assumir valores desatualizados guardados em registos. Devem evitar-se atrasos e operações lentas. A terceira regra é o erro mais comum de quem começa, e vale a pena tê-la presente no laboratório.

Diapositivo 18. O Hardware de Hoje (1/2)

O Hardware de Hoje (1/2)

- **Arduino Uno:** plataforma aberta de microcontrolador, hardware e software licenciados sob GPL/LGPL — a base de todos os exercícios de hoje.
- **LED onboard (pino 13) e botão de pressão:** para os dois exercícios de interrupções.
- **Sensor de humidade do solo (Funduino):** sensor resistivo analógico, lido em A0.

O equipamento, primeira parte. O Arduino Uno é uma plataforma aberta de microcontrolador. O LED integrado no pino 13 e o botão de pressão servem os dois exercícios de interrupções. O sensor de humidade do solo é resistivo e analógico.

Diapositivo 19. O Hardware de Hoje (2/2)

O Hardware de Hoje (2/2)

- **DHT11**: sensor digital de temperatura e humidade, com protocolo próprio de um único fio — é por isso que precisa de uma biblioteca dedicada (`dht.h`).
- **HC-SR04**: sensor ultrassónico de distância; mede o tempo de eco de um pulso sonoro para calcular a distância a um obstáculo.
- É apenas com este conjunto — o que temos disponível em laboratório — que o guião de hoje trabalha.

O equipamento, segunda parte. O DHT11 é digital e comunica por um protocolo próprio num único fio, razão pela qual exige uma biblioteca dedicada. O HC-SR04 mede distância pelo tempo de eco de um impulso sonoro. Estes dois sensores voltarão a aparecer na aula seguinte, quando a leitura passar a ser transmitida por rádio.

Diapositivo 20. Cuidados de Montagem

Cuidados de Montagem

- Montem sempre os circuitos com o Arduino **desligado do USB** (ou pelo menos sem código a correr que ative os pinos envolvidos).
- Confirmem a polaridade dos sensores antes de ligar: VCC e GND trocados podem danificar o componente.
- Não é preciso ter experiência prévia de eletrónica: cada exercício do guião indica exatamente que fios ligar a que pinos.

Antes de ligar. Os circuitos devem ser montados com a placa desligada. A polaridade deve ser confirmada antes de alimentar, porque alimentação e massa trocadas danificam o componente. Não é exigida experiência prévia de eletrónica: o guião indica exatamente que fios ligar a que pinos.

Diapositivo 21. Programação Visual: No-Code e Low-Code

Programação Visual: No-Code e Low-Code

- Nem toda a lógica de um sistema IoT precisa de ser escrita linha a linha: ferramentas **no-code** (sem código) e **low-code** (pouco código) permitem montar comportamentos ligando blocos visuais.
- Não substituem a programação — o Arduino de hoje continua a precisar do seu sketch — mas aceleram muito a prototipagem do que vem a seguir ao sensor: painéis, regras, integrações.
- É uma abordagem particularmente comum em IoT, onde é preciso ligar muitas peças diferentes (sensores, protocolos, bases de dados, notificações) rapidamente.

Programar sem escrever código. Nem toda a lógica de um sistema IoT precisa de ser escrita linha a linha. As ferramentas de programação visual permitem montar comportamentos ligando blocos. Não substituem a programação, e o Arduino continua a precisar do seu programa. Aceleram a prototipagem daquilo que vem a seguir ao sensor: painéis, regras e integrações.

Diapositivo 22. Node-RED

Node-RED

- O **Node-RED** é uma ferramenta open-source de programação visual baseada em *flows*: arrastam-se e ligam-se nós num navegador, sem escrever JavaScript.
- Cada nó representa uma pequena função — ler uma porta série, falar MQTT, desenhar um gráfico — e os fios entre eles representam o fluxo de dados.
- No laboratório de hoje o Node-RED é a ferramenta principal: com o firmware `StandardFirmata` carregado na placa, os pinos do Arduino passam a ser lidos e escritos diretamente a partir dos nós, sem escrever código.
- Só os sensores com protocolos temporizados (DHT11 e HC-SR04) exigem um *sketch* próprio, que vos será fornecido pronto a carregar.
- Voltaremos ao Node-RED na Aula 12, desta vez a receber dados por MQTT em vez de porta série.

O que é o Node-RED. O Node-RED é uma ferramenta open source de programação visual baseada em fluxos. Cada nó representa uma pequena função e os fios entre eles representam o percurso dos dados. Com o firmware genérico adequado carregado na placa, os pinos passam a ser lidos e escritos diretamente a partir dos nós. Voltar-se-á a esta ferramenta na Aula 12, então com um protocolo de mensagens em vez de ligação direta.

Diapositivo 23. Estrutura da Sessão Prática (2h)**Estrutura da Sessão Prática (2h)**

- Sete exercícios, todos com montagem física: LED, botão, sensor de humidade do solo, DHT11, HC-SR04 e um painel final que integra as várias leituras.
- Os três primeiros fazem-se inteiramente em Node-RED, sem escrever código; nos restantes, os *sketches* necessários são fornecidos.
- O exercício 6 volta ao nível do microcontrolador, para comparar o modelo orientado a eventos dos fluxos visuais com as interrupções por hardware e por software.

Como decorre a sessão. São sete exercícios, todos com montagem física. Os três primeiros fazem-se inteiramente em Node-RED, sem escrever código; nos restantes, os programas necessários são fornecidos. O sexto exercício regressa ao nível do microcontrolador, para comparar o modelo orientado a eventos dos fluxos visuais com as interrupções por hardware e por software. É aí que os dois blocos desta aula se encontram.

Diapositivo 24. Síntese da Aula

Síntese da Aula

- Sensores fazem a ponte entre o mundo analógico e o sistema digital; o ADC do Arduino Uno amostra essa grandeza em 10 bits, valores entre 0 e 1023.
- Interrupções, por hardware ou por software, permitem reagir a eventos sem desperdiçar ciclos em *polling* — desde que as ISRs sejam curtas e as variáveis partilhadas sejam *volatile*.
- Ferramentas low-code como o Node-RED permitem adquirir e visualizar esses dados sem escrever código, desde que o protocolo do sensor o permita — um tema a que voltaremos com MQTT na Aula 12.

Síntese da aula. Os sensores fazem a ponte entre o mundo analógico e o sistema digital, e o conversor do Arduino amostra em 10 bits, entre 0 e 1023. As interrupções permitem reagir a acontecimentos sem desperdiçar ciclos, desde que as rotinas sejam curtas e as variáveis partilhadas sejam declaradas *volatile*. As ferramentas de baixo código permitem adquirir e visualizar esses dados sem programar, desde que o protocolo do sensor o permita.

Verificação Final

Antes de dar a matéria por estudada, verifique se consegue responder ao seguinte sem consultar os diapositivos.

1. Distinguir sensor analógico de sensor digital e indicar a consequência dessa distinção na leitura.
2. Justificar por que motivo uma leitura do conversor do Arduino Uno se situa entre 0 e 1023.
3. Enunciar o critério de Nyquist e explicar por que não é restritivo para as grandezas do laboratório.
4. Comparar sondagem e interrupção quanto a consumo de processamento e a risco de perder eventos.
5. Explicar o papel da palavra `volatile` e o que sucede se for omitida.
6. Indicar as três regras a respeitar numa rotina de serviço de interrupção.
7. Explicar em que circunstâncias uma ferramenta de baixo código não dispensa a escrita de programa na placa.

Referências

- Gonçalves, P. *Apresent1 — Introdução* (bloco de sensores e conversão AD/DA) e *guia 1 — arduino*, TCIoT, edições anteriores.
- Documentação do Arduino sobre interrupções externas: arduino.cc/reference
- Documentação do Node-RED: nodered.org/docs/